For the problement statement, I am working on to Search for the official documentation of the German Bundestag's DIP API (Dokumentations- und Informationssystem für Parlamentsmaterialien). Find the base URL, authentication method, and a link to interactive API docs

Thought for 6s

Thought for 6s

Good — let's build the filtered "active/early-warning" mart on top of what you've already validated. Given the messy, inconsistent beratungsstand strings you just surfaced (15 distinct values, mixing genuinely open states like "Noch nicht beraten" with closed ones like "Erledigt durch Ablauf der Wahlperiode"), a hardcoded WHERE beratungsstand NOT IN (...) list buried in SQL is going to be hard to maintain and easy to get subtly wrong. A **dbt seed** (a CSV you control, joined in) is more transparent and easier to troubleshoot — exactly your stated goal.

dbt_project/seeds/dip_status_mapping.csv

csv

beratungsstand,status_category

Beantwortet,closed

Abgeschlossen - Ergebnis siehe Vorgangsablauf,closed

Abgelehnt,closed

Verkündet,closed

Nicht abgeschlossen - Einzelheiten siehe Vorgangsablauf,open

Erledigt durch Ablauf der Wahlperiode,closed

Für erledigt erklärt,closed

Angenommen,closed

Abgeschlossen,closed

Zusammengeführt mit... (siehe Vorgangsablauf),closed

Noch nicht beraten,open

Überwiesen,open

Zurückgezogen,closed

Beschlussempfehlung liegt vor,open

In der Beratung (Einzelheiten siehe Vorgangsablauf),open

This is your judgment call to review, not mine — I mapped based on the German phrasing (e.g. "Noch nicht beraten" = not yet debated = open; "Erledigt" = done = closed), but you should sanity-check each line against what a Public Affairs analyst would actually want flagged as "still moving."

dbt_project/dbt_project.yml — add a var for current Wahlperiode

yaml

```yaml
vars:

  current_wahlperiode: 21
```

Keeping this as a var (not hardcoded in the SQL) means next election period you change one line, not every model.

dbt_project/models/marts/mart_rentenpolitik_alerts.sql

sql

```sql
-- Early-warning view: current Wahlperiode + not-yet-closed Vorgänge only.

-- Sorted by most recently updated first — this IS the monitoring feed.


with base as (

   select * from {{ ref('mart_rentenpolitik_monitor') }}

),


status_mapped as (

   select

      b.*,

      coalesce(s.status_category, 'unknown') as status_category

   from base b

   left join {{ ref('dip_status_mapping') }} s
```

```sql
    on b.beratungsstand = s.beratungsstand
)

select
  vorgang_id,
  titel,
  vorgangstyp,
  beratungsstand,
  status_category,
  wahlperiode,
  datum,
  aktualisiert,
  initiativen,
  sachgebiete,
  abstract,
  dip_url,
  last_ingested_at,
  -- flag items updated in the last 14 days as "new activity" for the alert feed
  julianday('now') - julianday(aktualisiert) <= 14 as is_recent_update
from status_mapped
where wahlperiode = {{ var('current_wahlperiode') }}
  and status_category in ('open', 'unknown')  -- surface 'unknown' too, don't silently drop unmapped statuses
order by aktualisiert desc
```

Why status_category in ('open', 'unknown') rather than just 'open': if a new beratungsstand string appears in a future fetch that isn't yet in your seed CSV, coalesce turns it into 'unknown' rather than silently NULL-filtering it out of the join.

Surfacing unknown rows in the alert feed (rather than dropping them) means you'll *notice* an unmapped status and can add it to the seed — instead of quietly missing a real Vorgang because your mapping was incomplete. This is a deliberate fail-safe/fail-loud choice worth keeping given this is an early-warning system.

dbt_project/models/marts/schema.yml — add tests

yaml

```yaml
version: 2

models:
  - name: mart_rentenpolitik_alerts
    columns:
      - name: vorgang_id
        tests:
          - unique
          - not_null
      - name: status_category
        tests:
          - not_null
          - accepted_values:
              values: ['open', 'unknown']   # 'closed' should never appear here — tests the filter itself

seeds:
  - name: dip_status_mapping
    columns:
      - name: beratungsstand
        tests:
```

```
    - unique
    - not_null
  - name: status_category
    tests:
      - not_null
      - accepted_values:
          values: ['open', 'closed']
```

Run it

bash

```
dbt seed      # loads dip_status_mapping.csv into the db first
dbt run
dbt test
```

Once that's clean, pull it into pandas the same way as before:

python

```
df_alerts = pd.read_sql("SELECT * FROM mart_rentenpolitik_alerts", conn)
print(df_alerts.shape)
df_alerts[["titel", "beratungsstand", "status_category", "aktualisiert", "is_recent_update"]]
```

Given your earlier row-level check, expect this to shrink dramatically from 1,767 down to roughly single-to-low-double digits (the Noch nicht beraten: 4, Überwiesen: 3, In der Beratung: 1, Beschlussempfehlung liegt vor: 2 rows, all at WP 21, plus any unknown stragglers) — that small, high-signal list is exactly what an early-warning feed should look like. Run it and let me know what comes back.